Testing Plan

Name: Michael Pruett

Connection to My Problem Statement:

My project addresses the problem of the University of Evansville Swimming & Diving coaching staff with how they must manage scheduling information across multiple systems and manually compare swimmers' class schedules when determining practice times. The testing plan evaluates whether the platform can correctly identify conflicts, recommend low conflict practice times, and provide those results in a timely manner that will make it useful for the coaching staff.

Requirement 1 – User Requirement:

Requirement:

The coaching staff needs to be able to enter practice constraints and easily be able to identify practice times that work for the greatest number of swimmers.

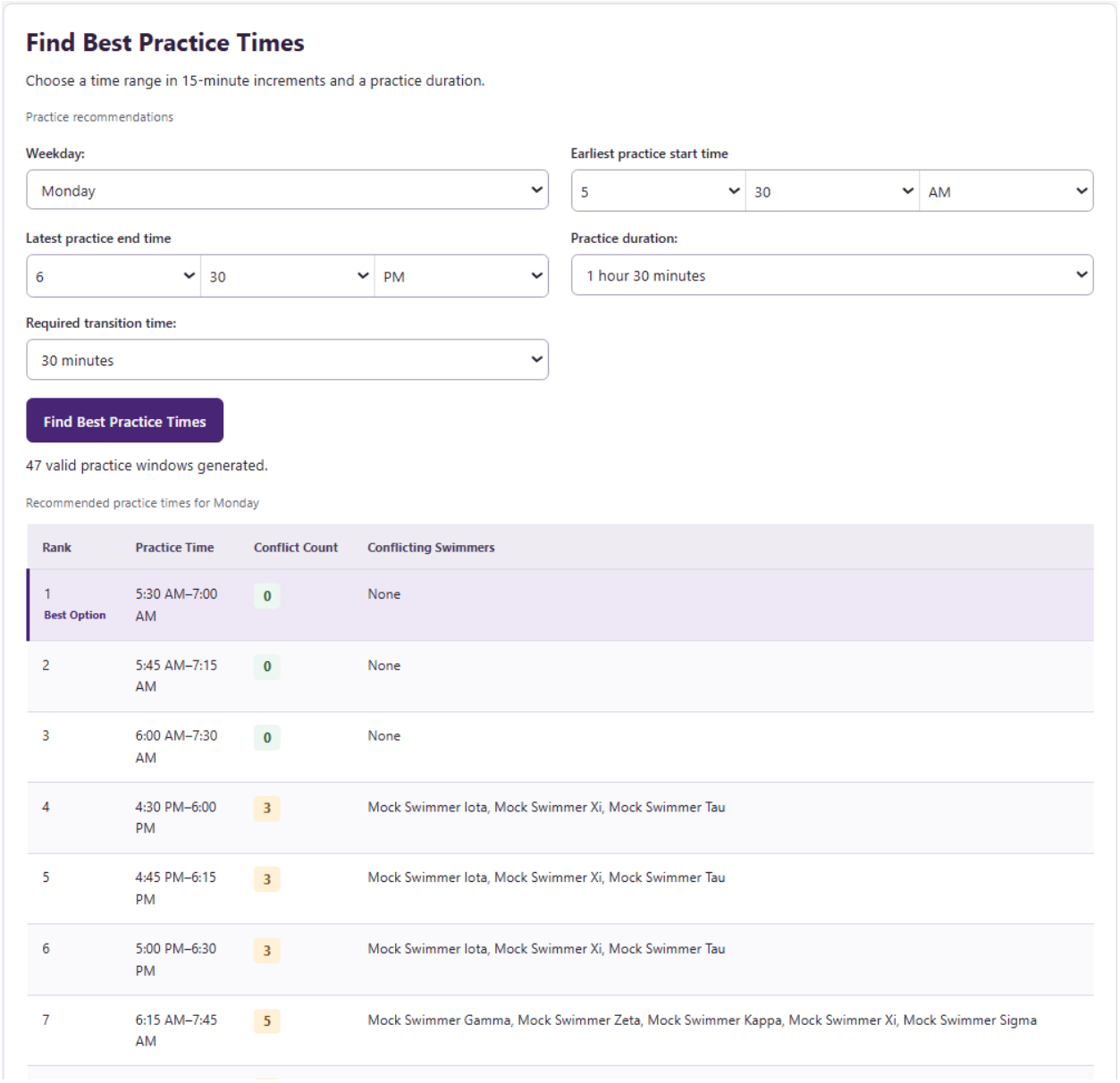
Test:

This is tested by the coach using the twenty fictional swimmer dataset and assuming they need to find a practice time. The coach will select the day of the week they want to find a practice for, enter in the earliest and latest allowed times, enter in the duration of practice time, and the transition time between practice and other responsibilities. The coach then can visibly see the generated recommendations and easily identify which times would work best for practice times on a certain day.

Expected Result:

The coach should be able to enter all of these constraints and get recommendation feedback that is accurate based on the available data.

Evidence:



Interpretation:

If the coach can complete the scheduling task and finds the recommendations useful, this would provide evidence that the system addresses the scheduling problem experienced by its intended users.

Requirement 2:

Requirement:

The system must be able to identify 100% of conflicts when schedules are entered correctly and then use these conflicts to rank generated practice windows by number of conflicts

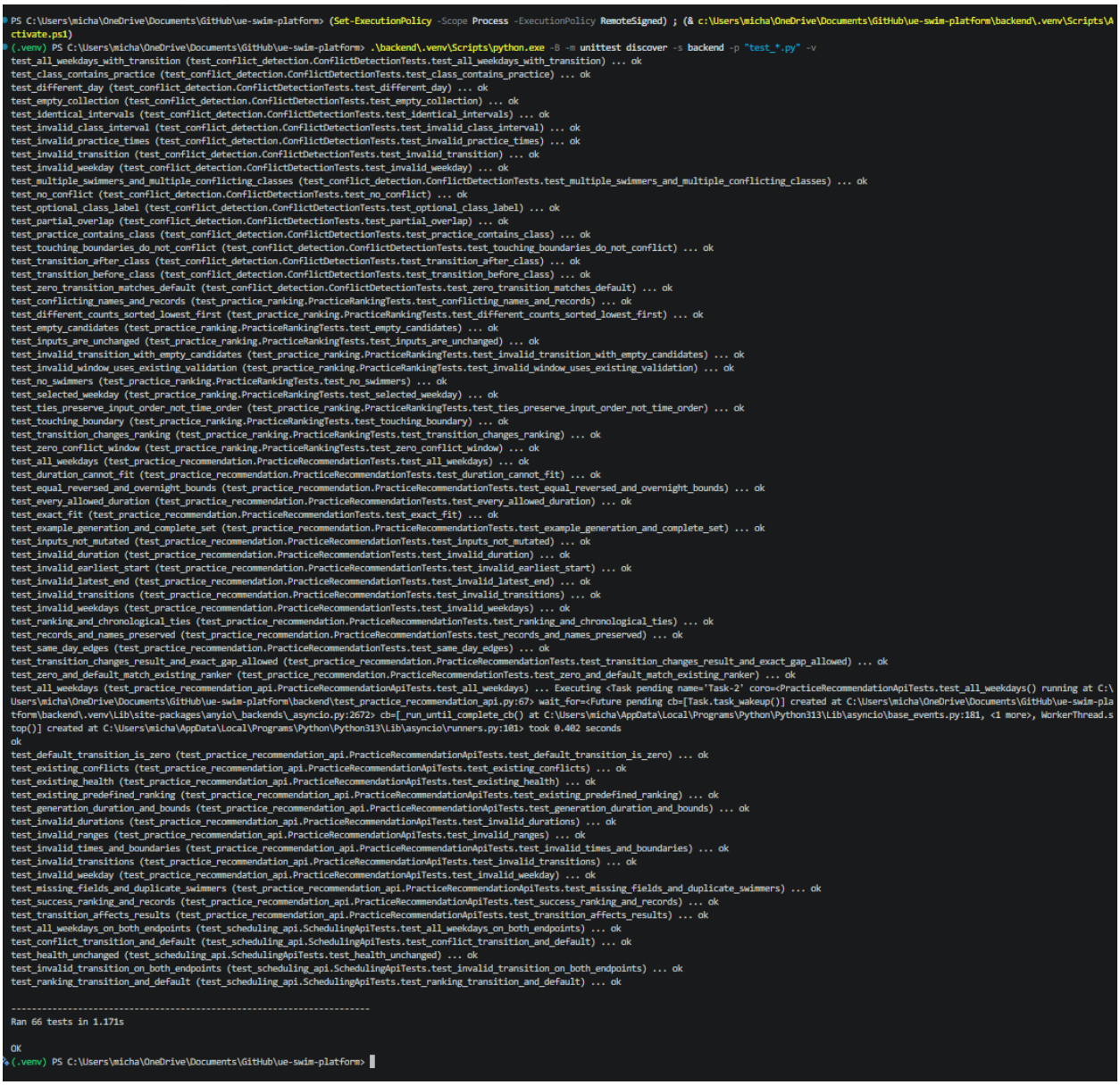
Test:

Several known scheduling cases will be created using the fictional swimmer schedules. These will include practices with no conflicts, direct class overlaps, multiple conflicts, exact time boundaries, and required transition times. The expected conflicts will be determined beforehand and compared with the results returned by the system.

Expected Result:

The system should identify every expected conflict correctly and rank the practice windows from lowest to highest based on the number of conflicts.

Evidence:



```
PS C:\Users\micha\OneDrive\Documents\GitHub\ue-swim-platform> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\micha\OneDrive\Documents\GitHub\ue-swim-platform\backend\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\micha\OneDrive\Documents\GitHub\ue-swim-platform> .\backend\.venv\Scripts\python.exe -B -m unittest discover -s backend -p "test_*.py" -v
test_all_weekdays_with_transition (test_conflict_detection.ConflictDetectionTests.test_all_weekdays_with_transition) ... ok
test_class_contains_practice (test_conflict_detection.ConflictDetectionTests.test_class_contains_practice) ... ok
test_different_day (test_conflict_detection.ConflictDetectionTests.test_different_day) ... ok
test_empty_collection (test_conflict_detection.ConflictDetectionTests.test_empty_collection) ... ok
test_identical_intervals (test_conflict_detection.ConflictDetectionTests.test_identical_intervals) ... ok
test_invalid_class_interval (test_conflict_detection.ConflictDetectionTests.test_invalid_class_interval) ... ok
test_invalid_practice_times (test_conflict_detection.ConflictDetectionTests.test_invalid_practice_times) ... ok
test_invalid_transition (test_conflict_detection.ConflictDetectionTests.test_invalid_transition) ... ok
test_invalid_weekday (test_conflict_detection.ConflictDetectionTests.test_invalid_weekday) ... ok
test_multiple_swimmers_and_multiple_conflicting_classes (test_conflict_detection.ConflictDetectionTests.test_multiple_swimmers_and_multiple_conflicting_classes) ... ok
test_no_conflict (test_conflict_detection.ConflictDetectionTests.test_no_conflict) ... ok
test_optional_class_label (test_conflict_detection.ConflictDetectionTests.test_optional_class_label) ... ok
test_partial_overlap (test_conflict_detection.ConflictDetectionTests.test_partial_overlap) ... ok
test_practice_contains_class (test_conflict_detection.ConflictDetectionTests.test_practice_contains_class) ... ok
test_touching_boundaries_do_not_conflict (test_conflict_detection.ConflictDetectionTests.test_touching_boundaries_do_not_conflict) ... ok
test_transition_after_class (test_conflict_detection.ConflictDetectionTests.test_transition_after_class) ... ok
test_transition_before_class (test_conflict_detection.ConflictDetectionTests.test_transition_before_class) ... ok
test_zero_transition_matches_default (test_conflict_detection.ConflictDetectionTests.test_zero_transition_matches_default) ... ok
test_conflicting_names_and_records (test_practice_ranking.PracticeRankingTests.test_conflicting_names_and_records) ... ok
test_different_counts_sorted_lowest_first (test_practice_ranking.PracticeRankingTests.test_different_counts_sorted_lowest_first) ... ok
test_empty_candidates (test_practice_ranking.PracticeRankingTests.test_empty_candidates) ... ok
test_inputs_are_unchanged (test_practice_ranking.PracticeRankingTests.test_inputs_are_unchanged) ... ok
test_invalid_transition_with_empty_candidates (test_practice_ranking.PracticeRankingTests.test_invalid_transition_with_empty_candidates) ... ok
test_invalid_window_uses_existing_validation (test_practice_ranking.PracticeRankingTests.test_invalid_window_uses_existing_validation) ... ok
test_no_swimmers (test_practice_ranking.PracticeRankingTests.test_no_swimmers) ... ok
test_selected_weekday (test_practice_ranking.PracticeRankingTests.test_selected_weekday) ... ok
test_ties_preserve_input_order_not_time_order (test_practice_ranking.PracticeRankingTests.test_ties_preserve_input_order_not_time_order) ... ok
test_touching_boundary (test_practice_ranking.PracticeRankingTests.test_touching_boundary) ... ok
test_transition_changes_ranking (test_practice_ranking.PracticeRankingTests.test_transition_changes_ranking) ... ok
test_zero_conflict_window (test_practice_ranking.PracticeRankingTests.test_zero_conflict_window) ... ok
test_all_weekdays (test_practice_recommendation.PracticeRecommendationTests.test_all_weekdays) ... ok
test_duration_cannot_fit (test_practice_recommendation.PracticeRecommendationTests.test_duration_cannot_fit) ... ok
test_equal_reversed_and_overnight_bounds (test_practice_recommendation.PracticeRecommendationTests.test_equal_reversed_and_overnight_bounds) ... ok
test_every_allowed_duration (test_practice_recommendation.PracticeRecommendationTests.test_every_allowed_duration) ... ok
test_exact_fit (test_practice_recommendation.PracticeRecommendationTests.test_exact_fit) ... ok
test_example_generation_and_complete_set (test_practice_recommendation.PracticeRecommendationTests.test_example_generation_and_complete_set) ... ok
test_inputs_not_mutated (test_practice_recommendation.PracticeRecommendationTests.test_inputs_not_mutated) ... ok
test_invalid_duration (test_practice_recommendation.PracticeRecommendationTests.test_invalid_duration) ... ok
test_invalid_earliest_start (test_practice_recommendation.PracticeRecommendationTests.test_invalid_earliest_start) ... ok
test_invalid_latest_end (test_practice_recommendation.PracticeRecommendationTests.test_invalid_latest_end) ... ok
test_invalid_transitions (test_practice_recommendation.PracticeRecommendationTests.test_invalid_transitions) ... ok
test_invalid_weekdays (test_practice_recommendation.PracticeRecommendationTests.test_invalid_weekdays) ... ok
test_ranking_and_chronological_ties (test_practice_recommendation.PracticeRecommendationTests.test_ranking_and_chronological_ties) ... ok
test_records_and_names_preserved (test_practice_recommendation.PracticeRecommendationTests.test_records_and_names_preserved) ... ok
test_same_day_edges (test_practice_recommendation.PracticeRecommendationTests.test_same_day_edges) ... ok
test_transition_changes_result_and_exact_gap_allowed (test_practice_recommendation.PracticeRecommendationTests.test_transition_changes_result_and_exact_gap_allowed) ... ok
test_zero_and_default_match_existing_ranker (test_practice_recommendation.PracticeRecommendationTests.test_zero_and_default_match_existing_ranker) ... ok
test_all_weekdays (test_practice_recommendation_api.PracticeRecommendationApiTests.test_all_weekdays) ... Executing <Task pending name='Task-2' coro=<PracticeRecommendationApiTests.test_all_weekdays() running at C:\Users\micha\OneDrive\Documents\GitHub\ue-swim-platform\backend\test_practice_recommendation_api.py:67> wait_for=<Future pending cb=[Task.task_wakeup()] created at C:\Users\micha\OneDrive\Documents\GitHub\ue-swim-platform\backend\.venv\Lib\site-packages\anyio\_backends\_asyncio.py:2672> cb=[_run_until_complete_cb() at C:\Users\micha\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py:181, <1 more>, WorkerThread.stop()] created at C:\Users\micha\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py:101> took 0.402 seconds
ok
test_default_transition_is_zero (test_practice_recommendation_api.PracticeRecommendationApiTests.test_default_transition_is_zero) ... ok
test_existing_conflicts (test_practice_recommendation_api.PracticeRecommendationApiTests.test_existing_conflicts) ... ok
test_existing_health (test_practice_recommendation_api.PracticeRecommendationApiTests.test_existing_health) ... ok
test_existing_predefined_ranking (test_practice_recommendation_api.PracticeRecommendationApiTests.test_existing_predefined_ranking) ... ok
test_generation_duration_and_bounds (test_practice_recommendation_api.PracticeRecommendationApiTests.test_generation_duration_and_bounds) ... ok
test_invalid_durations (test_practice_recommendation_api.PracticeRecommendationApiTests.test_invalid_durations) ... ok
test_invalid_ranges (test_practice_recommendation_api.PracticeRecommendationApiTests.test_invalid_ranges) ... ok
test_invalid_times_and_boundaries (test_practice_recommendation_api.PracticeRecommendationApiTests.test_invalid_times_and_boundaries) ... ok
test_invalid_transitions (test_practice_recommendation_api.PracticeRecommendationApiTests.test_invalid_transitions) ... ok
test_invalid_weekday (test_practice_recommendation_api.PracticeRecommendationApiTests.test_invalid_weekday) ... ok
test_missing_fields_and_duplicate_swimmers (test_practice_recommendation_api.PracticeRecommendationApiTests.test_missing_fields_and_duplicate_swimmers) ... ok
test_success_ranking_and_records (test_practice_recommendation_api.PracticeRecommendationApiTests.test_success_ranking_and_records) ... ok
test_transition_affects_results (test_practice_recommendation_api.PracticeRecommendationApiTests.test_transition_affects_results) ... ok
test_all_weekdays_on_both_endpoints (test_scheduling_api.SchedulingApiTests.test_all_weekdays_on_both_endpoints) ... ok
test_conflict_transition_and_default (test_scheduling_api.SchedulingApiTests.test_conflict_transition_and_default) ... ok
test_health_unchanged (test_scheduling_api.SchedulingApiTests.test_health_unchanged) ... ok
test_invalid_transition_on_both_endpoints (test_scheduling_api.SchedulingApiTests.test_invalid_transition_on_both_endpoints) ... ok
test_ranking_transition_and_default (test_scheduling_api.SchedulingApiTests.test_ranking_transition_and_default) ... ok

----------------------------------------------------------------------
Ran 66 tests in 1.171s

OK
(.venv) PS C:\Users\micha\OneDrive\Documents\GitHub\ue-swim-platform>
```

Interpretation:

The backend tests passed for the known scheduling and conflict cases, producing the expected results. This supports the problem statement's target of identifying 100% of conflicts when schedules are entered correctly.

Requirement 3:

Requirement:

The system should be able to rank and generate practice windows based on conflicts in 10 seconds or less.

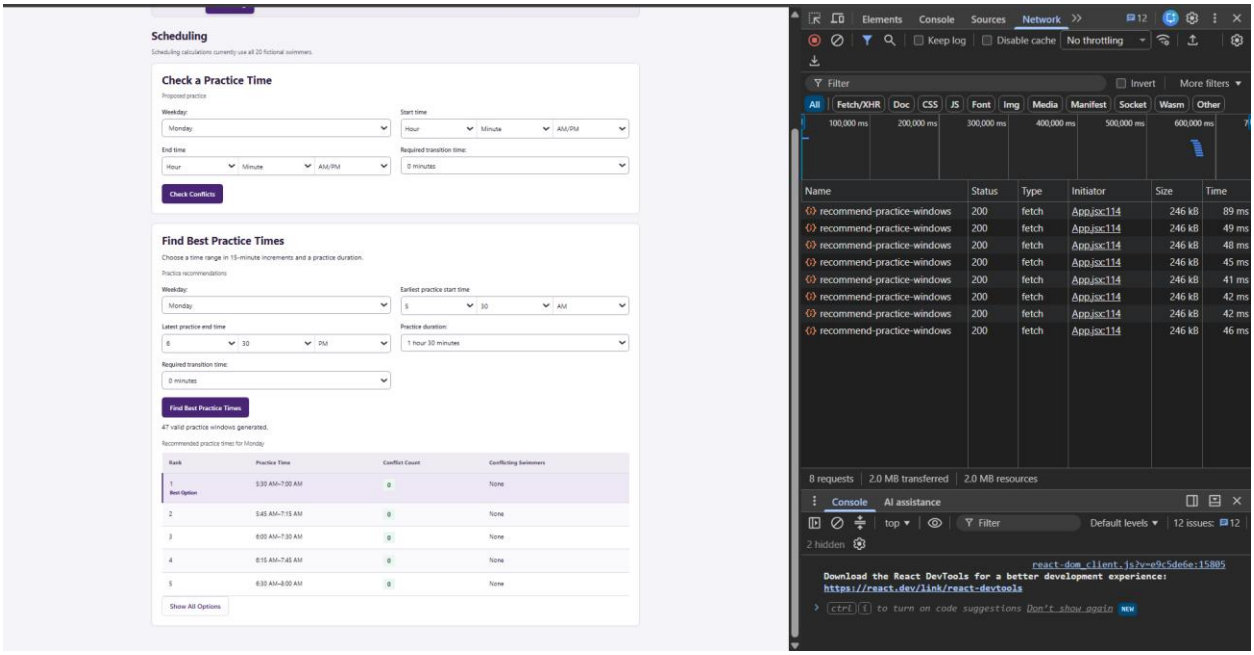
Test:

The practice recommendation feature will be tested using all 20 fictional swimmers and several combinations of weekdays, allowable time ranges, practice durations, and transition times. The total time from requesting recommendations until the ranked results are available to the user will be recorded across multiple trials.

Expected Result:

Every test should return the ranked practice recommendations in 10 seconds or less.

Evidence:



Interpretation:

The practice recommendation request shown in the test completed in 89 milliseconds, or 0.089 seconds. This is well below the 10-second target established in the problem statement and provides evidence that the current prototype meets the performance requirement.

Validation Method:

The main validation method I will use for my project is feedback and task-completion observation from the University of Evansville Swimming & Diving coaching staff. I communicate with my client regularly and use his feedback to tailor the system to what he is looking for. My coach has used the prototype and provided feedback multiple times, and I plan to continue this. Comments and observations will be recorded to determine whether the system addresses the real problem rather than only functioning correctly from a technical standpoint.

Performance Measure:

The primary performance measure will be recommendation response time. The time required to generate and rank practice windows will be measured and compared with the target of 10 seconds or less from the original problem statement. Conflict-detection accuracy will also be recorded, with a target of 100% accuracy for correctly entered test schedules.